



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

**Faculty of Engineering and Information Sciences
School of Computing & Information Technology**

CSIT321: Project
Capstone Project - Spring 2025

Initial Project Description

Team Members

Noor Ahmed - 7295297

Nishad Gyawali - 8124553

Chelsea Okan - 8438675

University of Wollongong
Liverpool Campus

Subject Coordinator
Dr. John Le

27th March, 2026

Table of Contents

Prototype Link	1
Participation Table	1
1. Project Overview	2
2. Project Requirements	3
2.1 Target Users and Design Goals	3
2.2 Functional Requirements	3
2.3 Non-Functional Requirements	4
3. System Architecture and Technology Design	5
3.1 Logical Architectural Layers	5
3.2 Technology Stack	11
4. Information Architecture	12
4.1 Navigation Structure	12
4.2 Core Information Objects	13
4.3 API and Workflow Mapping	14
5. Interface Design	16
5.1 Screen Inventory and Rationale	17
5.2 Interface Principles	21
6. Branding and Style Guide	22
6.1 Brand Positioning and Identity	22
6.2 Visual Style Guide	22
6.3 Application to the Prototype	24
7. Functionality and User Interaction	25
7.1 Primary Interaction Sequence	25
7.2 Primary Scenario Walkthrough	26
8. Prototype Progress	27
9. Current Limitations and Next Iteration Work	29
10. Project Risks and Progress Management	31
11. Conclusion	32
Appendix A. Additional Interface Screens	33
Appendix B. Supervisor Progress/Comments	36
References	37

Prototype Link

The working prototype for SmartResearch is available via the project repository:

<https://github.com/auweii/smartresearch>

The repository contains all current source code, including the frontend (React), backend (FastAPI), and processing pipeline. The system has been tested on Python 3.11.14, and instructions for setup and execution are provided in the README.

Contribution

Chelsea Okan

Overall Contribution

Area	Contribution
Project Coordination	Coordinated overall project progress, including task organisation and submission preparation
Repository Management	Set up and maintained the GitHub repository as the central workspace for code and documentation
Communication	Managed meeting documentation and communication with the project advisor
Frontend Development	Designed and implemented the frontend structure, including routing, UI flow, and integration with backend endpoints
Documentation	Led the writing, integration, and final editing of project documentation across all submissions
Backend Support	Supported backend development and integration across text extraction, summarisation, search, clustering, and metadata handling
System Alignment	Ensured alignment between the implemented system and the documentation presented in the report

This submission

Area	Contribution
Documentation	Wrote the full Design Progress Documentation (Assignment 4)
Diagrams	Developed and finalised activity, sequence, and data/storage diagrams
Interaction Design	Wrote and structured the Primary Interaction Sequence and supporting design sections
Technical Alignment	Reviewed and aligned technical descriptions with the implemented system
Submission Structure	Structured appendix content, figure captions, and cross-referencing

Noor Ahmed**Overall Contribution**

Area	Contribution
Backend Contribution	Contributed updates to application logic, schemas, and supporting services
Design Support	Assisted with design-related components and drafting support during earlier stages

This submission

Area	Contribution
Presentation	Prepared presentation slides and supporting notes
Visual Communication	Contributed to presentation of system functionality

Nishad Gyawali**Overall Contribution**

Area	Contribution
Presentation	Participated in presentation delivery during project assessments
Demo Recording	Recorded the prototype demonstration video

This submission

Area	Contribution
Presentation	Contributed to presentation delivery

1. Project Overview

SmartResearch is an AI-assisted literature review support platform designed to reduce the time and effort involved in screening academic papers. As established in the initial project requirements, the project addresses several common challenges in research workflows, including large volumes of documents, inconsistent note-taking, and limited visibility over thematic relationships across a paper set. SmartResearch responds to these issues by enabling users to upload batches of academic PDFs, generate concise structured summaries, and organise papers into interpretable topic clusters, making it easier to move from initial screening to synthesis.

Phase 1 is implemented as a bounded web-based system focused on demonstrating the core workflow of uploading English, text-selectable PDFs, extracting and storing text locally, generating metadata and summary outputs, clustering papers, supporting search and browsing, and exporting structured results for downstream review. The current prototype reflects this scope while also extending the system with selectable search modes, richer paper detail views, deletion controls, and PDF export. These additions strengthen the overall system and ensure the design remains aligned with the implemented functionality.

2. Project Requirements

This section maps the previously defined requirements to the current system design and prototype implementation.

2.1 Target Users and Design Goals

Target Users	Design Goal
Students and novice researchers	Reduce the time required to sort, scan, and revisit collections of academic papers
Research teams working with many PDFs	Support consistent upload, summarisation, grouping, and later retrieval of a paper corpus
Supervisors and advisors	Make outputs easier to inspect, review, and validate against project expectations
Future students/institutions	Create reusable structured outputs and a workflow that can be demonstrated, taught and extended

2.2 Functional Requirements

id	Requirement	Design Response	Prototype Status
FR1	Batch upload of at least 10 English, text-selectable PDF papers	Upload workflow with validation, queueing, and backend ingestion pipeline	Implemented / refinement required – upload and processing pipeline exists (text extraction + OCR), but batch reliability at ≥ 10 papers and validation behaviour require further testing
FR2	Generate structured summaries under 120 words using four fields: objective, method, findings, limitations	Summary model and storage designed around fixed summary sections for consistency	Implemented / refinement required – extractive and abstractive summarisation are present, but strict enforcement of structure and length is not yet guaranteed
FR3	Cluster papers using unsupervised methods with automatic keyword or keyphrase labels	Backend clustering workflow with labelled topic groupings and cluster exploration view	Implemented / refinement required – clustering pipeline exists, but output quality and keyword labelling require improvement
FR4	Provide interface views for upload, browse, cluster, and export	React routing and dedicated page structure map one screen to each workflow stage	Implemented – frontend routing and page structure are complete and connected across all main workflows
FR5	Support search and filtering when reviewing processed papers.	Browse / All Papers view exposes searchable metadata and summary access.	Implemented / refinement required – search modes and metadata browsing are present, but detail views and interaction elements require completion

FR6	Export structured results for reuse outside the system.	Export view and backend export logic support downstream analysis workflows.	Implemented / refinement required – export functionality exists, but end-to-end selection and formatting behaviour requires validation
NFR1	Process a 10-paper batch in 60 seconds or less where feasible for phase 1.	Lightweight local architecture, bounded prototype scope, and backend processing pipeline.	Target being validated - pipeline supports batching, but no confirmed benchmarks; performance may be affected by OCR, summarisation, and vector generation.
NFR2	Keep any paper reachable in 3 clicks or fewer.	Shallow navigation with four top-level views and predictable page layout.	Designed / partially met – navigation structure supports this, but incomplete UI elements limit full verification
NFR3	Handle malformed files gracefully without crashing the application.	Validation, error reporting, and bounded phase 1 file assumptions.	Implemented / refinement required – error handling exists, but robustness across edge cases is not fully tested
NFR4	Preserve privacy through local-only processing.	Local storage and no required third-party transfer for core workflow	Partially met – core processing is local, but optional external metadata enrichment introduces limited external dependency

2.3 Non-Functional Requirements

Quality Area	Requirement	Design Response
Usability	A novice user should be able to locate a known or related paper in 3 clicks or fewer	Persistent top-level navigation, a four-view workflow, and simple search and browsing patterns support quick access to documents
Performance	A 10-paper batch should process in around 60 seconds for the phase 1 target.	Lightweight local architecture, constrained prototype scope, and file-based storage reduce processing overhead
Reliability	Malformed or invalid PDFs should not crash the application	Input validation, visible error states, and controlled assumptions around supported PDF types maintain system stability
Privacy	Files should remain local to the system	Local-only processing and storage form the baseline, with any external enrichment treated as optional
Maintainability	The codebase should remain understandable within the assigned project timeframe	Clear frontend/backend separation, modular services, and repository-based documentation support maintainability

3. System Architecture and Technology Design

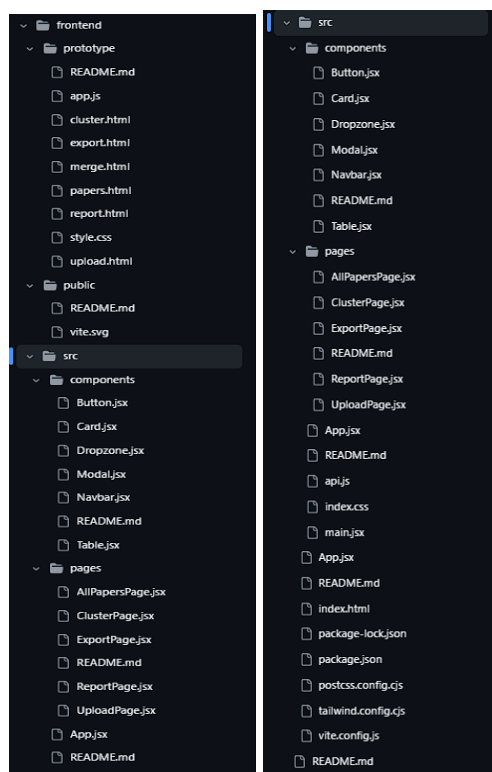
The system follows a lightweight three-tier web architecture consisting of a React frontend for user interaction, a Python FastAPI backend for processing and application logic, and a local storage layer for uploaded papers, extracted text, and generated metadata. This separation supports parallel development, keeps the interface straightforward, and isolates heavier tasks such as parsing, summarisation, clustering, search, and export within the backend.

Architecture Overview

Users upload PDF files through the React interface, which sends requests to FastAPI endpoints for processing. The backend handles text extraction, metadata generation, summarisation, clustering, and storage of generated outputs. Once processed, results become available through the All Papers, Cluster, and Export views for browsing, analysis, and reuse.

3.1 Logical Architectural Layers

Layer	Major Elements	Responsibility
Presentation Layer	React, Vite, Tailwind CSS, React Router, reusable components and page modules	Renders the core user workflow across the Upload, All Papers, Cluster, and Export views while maintaining clear, shallow navigation



Frontend repository structure showing the presentation layer of the system. This layer contains the user interface implementation, including reusable components, page-level views, and application entry files that support the main workflow screens such as upload, all papers, clustering, and export

Application Layer	FastAPI app, models, services, utilities, ReportLab-based export generation	Coordinates document ingestion, metadata retrieval, search, clustering, export, and other document-level actions
Processing Layer	PDF extraction with OCR fallback, embedding-based semantic retrieval, k-means clustering, summarisation logic, and recommendation logic	Performs the analysis and transformation tasks that convert uploaded documents into structured, searchable, and grouped research outputs ready for exploration and export



Backend repository structure representing the application layer: the backend is organised into models (data schemas), services (core processing logic), and utilities (supporting functions), with app.py serving as the api entry point.

```
smartresearch / backend / services / pdf_processing.py

noorahmedraza13098-dotcom Create pdf_processing.py

Code Blame 28 lines (22 loc) · 820 Bytes

1 # services/pdf_processing.py
2
3 from services.extract import pdf_to_text
4 from services.summarize import textrankish_summary
5 from services.metadata import enrich_from_text
6
7 def process_pdf(file_path: str, max_summary_sentences: int = 5) -> dict:
8     """
9     Extract text, generate summary, and enrich metadata.
10    Returns a dict suitable for storing in your database.
11    """
12    text = pdf_to_text(file_path)
13
14    if not text.strip():
15        text = "No readable content found in PDF."
16
17    # Extractive summary
18    summary = textrankish_summary(text, max_sentences=max_summary_sentences)
19
20    # Metadata enrichment
21    meta = enrich_from_text(text) or {}
22
23    return {
24        "full_text": text,
25        "summary": summary,
26        "abstract": summary, # optional: reuse summary for abstract
27        "meta": meta
28    }
```

Document processing pipeline coordinating extraction, summarisation, metadata enrichment, and indexing into a unified workflow for each uploaded pdf.

```
smartresearch / backend / services / extract.py

auweli Update extract.py

Code Blame 30 lines (25 loc) · 876 Bytes

1 from PyPDF2 import PdfReader
2 from pdf2image import convert_from_path
3 import pytesseract
4
5 def pdf_to_text(path: str) -> str:
6     """
7     Extract selectable text directly from a PDF.
8     If a page has no selectable text, use OCR instead.
9     If pages fail, skip them gracefully.
10    """
11    reader = PdfReader(path)
12    chunks = []
13
14    for i, page in enumerate(reader.pages):
15        try:
16            text = (page.extract_text() or "").strip()
17        except Exception:
18            text = ""
19
20        if not text:
21            try:
22                images = convert_from_path(path, first_page=i + 1, last_page=i + 1)
23                text = pytesseract.image_to_string(images[0]).strip() if images else ""
24            except Exception:
25                text = ""
26
27        if text:
28            chunks.append(text)
29
30    return "\n".join(chunks).strip()
```

Text extraction module responsible for parsing pdf content and applying ocr fallback for scanned documents, forming the initial input for downstream processing

```

smartresearch / backend / services / summarize.py
noorahmedraza13098-dotcom Update summarize.py

Code Blame 36 lines (31 loc) · 1.17 KB

1 import re
2 from typing import List
3 from nltk.corpus import stopwords
4 from nltk.tokenize import sent_tokenize
5 import nltk
6 import os
7
8 nltk.data.path.append(os.path.expanduser("~/nltk_data"))
9
10 def textrankish_summary(text: str, max_sentences: int = 5) -> str:
11     """
12     A lightweight extractive summarizer inspired by TextRank.
13     Scores sentences by term frequency and selects the top few.
14     """
15     sents = sent_tokenize(text)
16     if len(sents) <= max_sentences:
17         return text
18
19     # count non-stopword frequencies
20     words = re.findall("[A-Za-z]{2,}", text.lower())
21     sw = set(stopwords.words("english"))
22     freq = {}
23     for w in words:
24         if w not in sw:
25             freq[w] = freq.get(w, 0) + 1
26
27     # crude scoring: sum of word frequencies per sentence
28     scored = [
29         (sum(freq.get(w.lower(), 0) for w in re.findall("[A-Za-z]{2,}", s)) / (len(s.split()) + 1), s)
30         for s in sents
31     ]
32
33     # take top N sentences by score, restore original order
34     top = sorted(scored, key=lambda t: t[0], reverse=True)[max_sentences:]
35     ordered = [s for _, s in sorted(top, key=lambda x: sents.index(x[1]))]
36     return " ".join(ordered)

```

Summarisation module generating structured summaries from extracted text, supporting both extractive and abstractive methods.

```

smartresearch / backend / services / cluster.py
auweil Add files via upload

Code Blame 17 lines (13 loc) · 543 Bytes

1 from sklearn.cluster import KMeans
2
3 class Clusterer:
4     """
5     Wraps sklearn KMeans for simple clustering of embedded document vectors.
6     """
7
8     def __init__(self, k=4, random_state=42):
9         self.km = KMeans(n_clusters=k, random_state=random_state, n_init=10)
10
11     def fit(self, X):
12         """Fit and return predicted cluster labels."""
13         return self.km.fit_predict(X).tolist()
14
15     def centroids(self):
16         """Return cluster centroids as Python lists."""
17         return self.km.cluster_centers_.tolist()

```

Clustering component applying k-means to embedded document vectors to group semantically similar documents.

```
smartresearch / backend / services / semantic.py
auweil Add files via upload

Code Blame 244 lines (281 loc) · 7.57 KB

1 import os, json
2 from pathlib import Path
3 from typing import List, Tuple, Dict
4 import numpy as np
5 from sentence_transformers import SentenceTransformer
6
7 # optional FAISS acceleration (used if installed)
8 try:
9     import faiss
10     _HAS_FAISS = True
11 except Exception:
12     _HAS_FAISS = False
13
14 # config / globals
15 DATA_DIR = Path(os.getenv("SMARTRESEARCH_DATA", "/data_store"))
16 SEM_FILE = DATA_DIR / "semantic_chunks.json"
17
18 EMB_MODEL_NAME = os.getenv("SR_EMB_MODEL", "allenai/specter2_base")
19 _model = SentenceTransformer(EMB_MODEL_NAME)
20
21 # in-memory store (aka the semantic swap)
22 _ids: List[str] = [] # chunk IDs like "doc1:0"
23 _doc_lookup: Dict[str, str] = {} # chunk_id → doc_id
24 _vecs: np.ndarray = np.zeros(
25     (_model.get_sentence_embedding_dimension(), dtype="float32")
26 )
27
28 _index = None
29
30 # internal helpers
31
32 def _save():
33     """Persist chunk embeddings to disk."""
34     DATA_DIR.mkdir(parents=True, exist_ok=True)
35     payload = {"ids": _ids, "vecs": _vecs.tolist(), "lookup": _doc_lookup}
36     SEM_FILE.write_text(json.dumps(payload), encoding="utf-8")
37
38
39 def _rebuild_index():
40     """Rebuild FAISS or Numpy index depending on what's available."""
41     global _index
42     if _vecs.shape[0] == 0:
43         _index = None
44         return
45
46     if _HAS_FAISS:
47         dim = _vecs.shape[1]
48         ix = faiss.IndexFlatIP(dim)
49         normed_vecs = _vecs / np.linalg.norm(_vecs, axis=1, keepdims=True)
50         ix.add(normed_vecs.astype("float32"))
51         _index = ix
52     else:
53         _index = None
54
```

Semantic processing module implementing embedding-based retrieval using sentence-transformers and similarity search, enabling semantic matching between documents and queries.

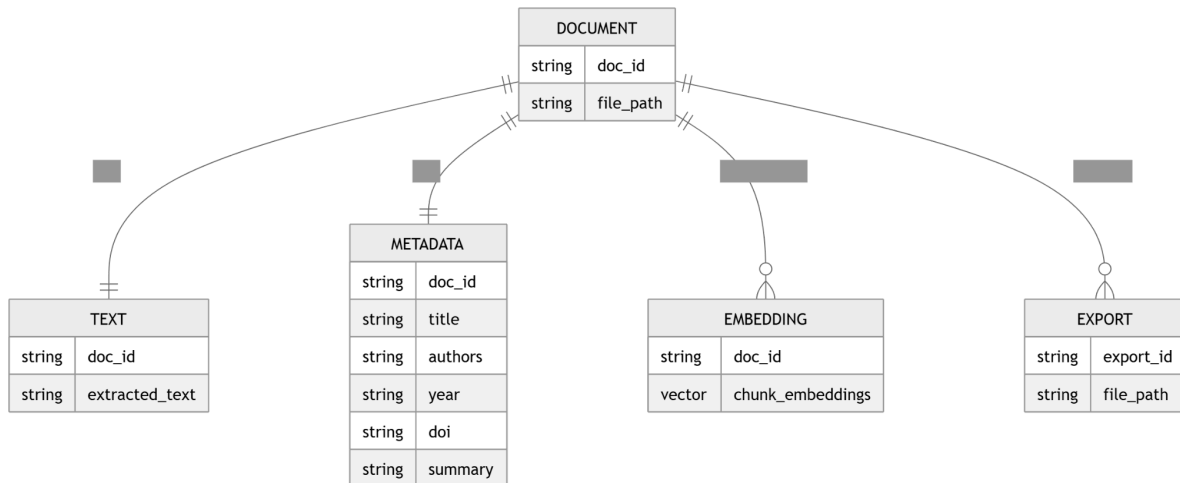
```
auweil Add files via upload

Code Blame 73 lines (63 loc) · 2.62 KB

1 import re
2 import requests
3
4 def enrich_from_text(text: str):
5     """
6     Try to extract bibliographic metadata from the first few lines of a document
7     using the CrossRef API. Adds sanity checks to avoid misclassifying
8     random content (like gym guides or cookbooks) as academic work.
9     """
10    lines = [l.strip() for l in text.splitlines() if l.strip()]
11    if not lines:
12        return None
13
14    # quick reject for clearly non-academic stuff
15    lower_text = text.lower()
16    banned_terms = ["recipe", "calorie", "workout", "gym", "cookbook", "nutrition", "meal prep"]
17    if any(k in lower_text for k in banned_terms):
18        return None
19
20    for line in lines[:10]:
21        # skip obvious junk (copyright, ISBNs, etc.)
22        if re.search(r"(copyright|all rights reserved|reproduction|isbn|contents)", line, re.I):
23            continue
24        if len(line.split()) < 4:
25            continue
26
27        try:
28            # ask CrossRef who the hell wrote this thing
29            r = requests.get(
30                "https://api.crossref.org/works",
31                params={"query.bibliographic": line, "rows": 1},
32                timeout=10,
33            )
34            data = r.json().get("message", {}).get("items", [])
35            if not data:
36                continue
37
38            it = data[0]
39            meta = {
40                "title": (it.get("title") or [None])[0],
41                "authors": [
42                    f"{a.get('given','')} {a.get('family','')}".strip()
43                    for a in it.get("author", [])
44                ],
45                "year": it.get("issued", {}).get("date-parts", [[None]])[0][0],
46                "doi": it.get("DOI"),
47                "venue": (it.get("container-title") or [None])[0],
48                "publisher": it.get("publisher"),
49            }
50
51            # sanity filters - because not everything CrossRef returns is sane
52            if not meta["title"]:
53                return None
54            if meta["title"].strip().lower() in {
55                "copyright", "untitled", "contents", "index", "table of contents"
56            }:
57                return None
58
59            # basic DOI format validation
60            doi = meta.get("doi")
61            if doi and not re.match(r"^\d\.\d(4,9)/[-_;()/:A-Za-z0-9]+$", doi):
62                meta["doi"] = None
63
64            # skip false positives
65            if meta["publisher"] in {"Elsevier", "Springer"} and "copyright" in lower_text:
66                return None
67
68            return meta
69        except Exception:
70            # API can be flaky, don't die on one bad query
71            continue
72
73    return None
```

Metadata processing module extracting, enriching, and structuring document metadata for indexing and display.

Storage Layer	Local file-based storage for uploaded PDFs, extracted text, metadata, semantic indices, and generated exports	Provides lightweight persistence for uploaded files and processed outputs across the system.
---------------	---	--



Data model diagram showing the file-based storage structure for documents, extracted text, metadata, embeddings, and export outputs

3,2 Technology Stack

Area	Technology	Reason for Use
Frontend	React, Vite, Tailwind CSS	Supports fast iteration, a clear component structure, and consistent interface styling
Routing and Client Calls	React Router, Axios	Maps the workflow across route-based views and enables communication with backend API endpoints
Backend	FastAPI (Python)	Provides a lightweight API layer suited to document processing and backend orchestration
Text Extraction	PyPDF2 with OCR fallback via pytesseract and pdf2image	Supports text-selectable PDFs and provides a fallback for scanned documents
Search and Clustering	scikit-learn, SentenceTransformers, FAISS	Enables TF-IDF search, semantic retrieval, clustering, and related discovery workflows
Export Generation	ReportLab with API-driven file responses	Supports generation of downloadable reports and structured export outputs
Storage	Local filesystem with JSON indexing	Provides lightweight, offline-friendly persistence for documents and metadata

Governance	GitHub repository, documentation, risk register, acceptance criteria, meeting minutes	Maintains traceability between design work, prototype progress, and submission evidence
------------	---	---

4. Information Architecture

4.1 Navigation Structure

The information architecture remains intentionally shallow because SmartResearch is designed as a focused workflow tool rather than a broad research dashboard. Users move through a small number of predictable views that reflect the research process itself: uploading papers, reviewing the corpus, exploring clustered themes, and exporting relevant outputs. Within this structure, the prototype also supports secondary actions such as detail modals, search mode switching, open-PDF actions, delete controls, and optional external recommendations. These features extend the system's functionality without disrupting the clarity of the core four-view workflow.

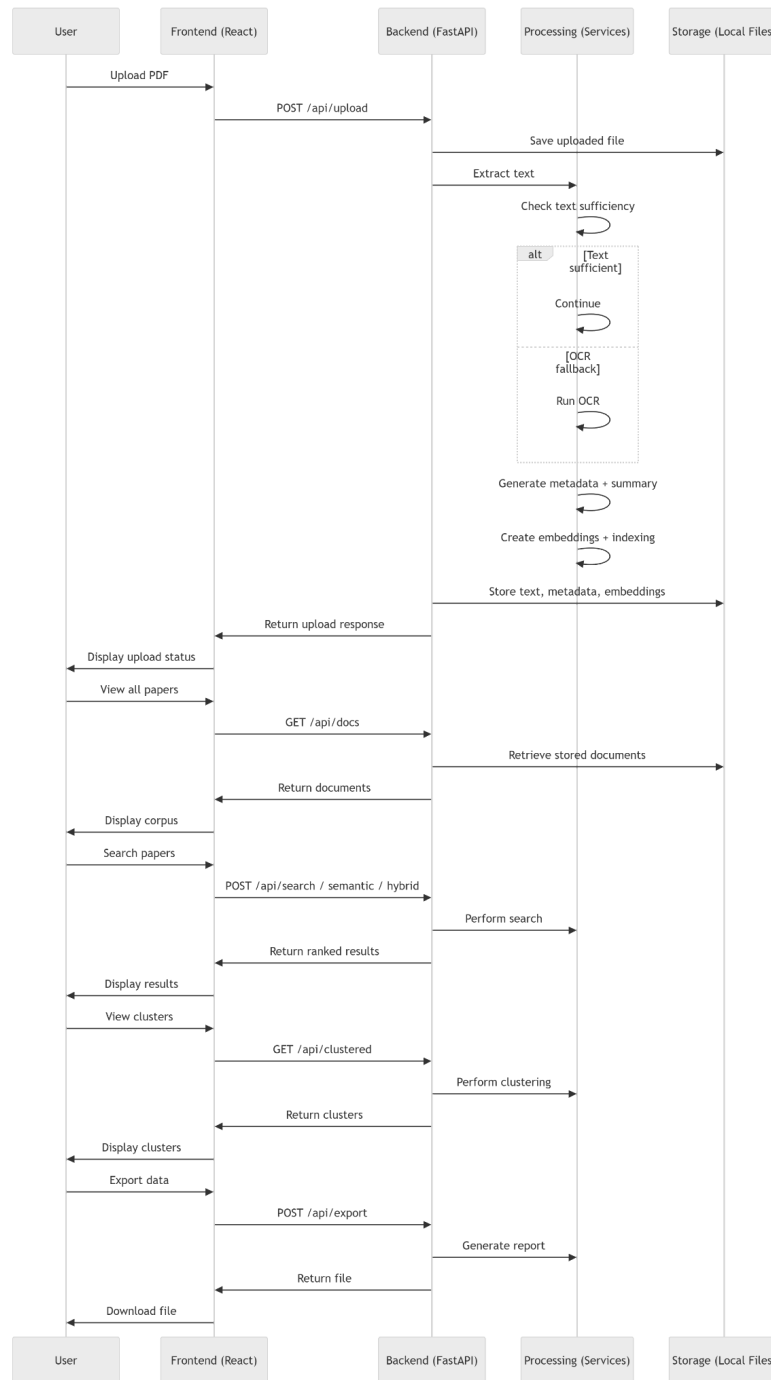
View	Primary Purpose	Key User Action
Upload	Entry point for new documents	Upload PDF files through drag-and-drop or file selection and begin processing
Browse / All Papers	Corpus browsing and retrieval	Search the paper set, inspect metadata, and view stored paper details and summaries
Cluster	Theme exploration	View grouped papers and explore clustered topics with associated keywords
Export	Reuse outside the system	Generate and download reports or structured outputs for downstream use

4.2 Core Information Objects

Object	Stored / Displayed Fields	Used By
Document	ID, name, file path, extracted text, and stored content	Upload, Browse / All Papers, Search, Cluster, Export
Metadata	Title, authors, year, abstract, DOI, publisher, and summary data	Paper details, previews, search results, export, recommendations
Cluster	Cluster ID, label, paper count, keywords, grouped document references	Cluster overview and grouped browsing
Search Hit	Document ID, name, relevance score, preview snippet	Keyword, semantic, and hybrid search results
Export Report	Selected content, grouped papers, summaries, keywords, structured output sections	Downloadable report and export outputs

4.3 API and Workflow Mapping

Method	Endpoint	Purpose
GET	/api/health	Check service availability
GET	/api/docs	Retrieve indexed documents with available summaries
GET	/api/text/{doc_id}	Retrieve full stored text for a document
GET	/api/meta/{doc_id}	Retrieve detailed metadata for a document
POST	/api/upload	Upload and process a PDF document
POST	/api/move_to_storage	Move uploaded documents into the main corpus
DELETE	/api/docs/{doc_id}	Delete a document and its associated data
GET	/api/clustered	Retrieve clustered document groups
POST	/api/search	Perform keyword search
POST	/api/semantic_search	Perform semantic search using embeddings
POST	/api/hybrid_search	Perform weighted hybrid search
GET	/api/similar/{doc_id}	Retrieve locally similar documents
GET	/api/external_recs/{doc_id}	Retrieve optional external recommendations
POST	/api/export	Generate export output (PDF, CSV, or JSON)
POST	/api/reindex	Rebuild semantic embeddings from stored documents
GET	/files/{file_id}.pdf	Serve a stored PDF file



Sequence diagram showing interactions between the user, frontend, backend, processing services, and storage across document upload, processing, retrieval, clustering, and export

5. Interface Design

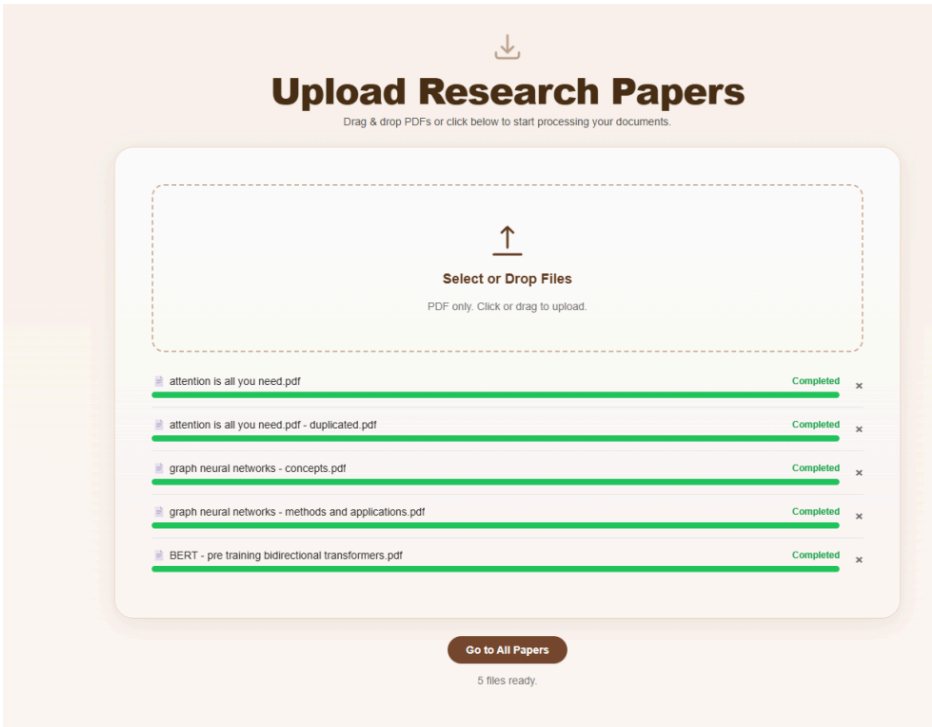
The interface has progressed from initial wireframes into a functioning prototype. The original wireframes established the core structure of the system, including the four-view workflow, shallow navigation, restrained bronze-accent styling, and the use of cards, tables, and document-focused layouts. The current prototype retains this structure while extending it with working interactions across the main workflow. Users can interact with multiple search modes, view detailed paper information, perform document-level actions, browse clustered themes, and access the export interface. These interactions reflect the intended user experience in a practical and testable form, rather than remaining at the level of layout design.

5.1 Screen Inventory and Rationale

Screen	Main Components	Design Intent
Upload View	Hero heading, upload card, drag-and-drop area, upload progress and status feedback, file preview modal, transition to corpus	Make the initial task clear and keep document ingestion low-friction while providing visible feedback on upload and processing state

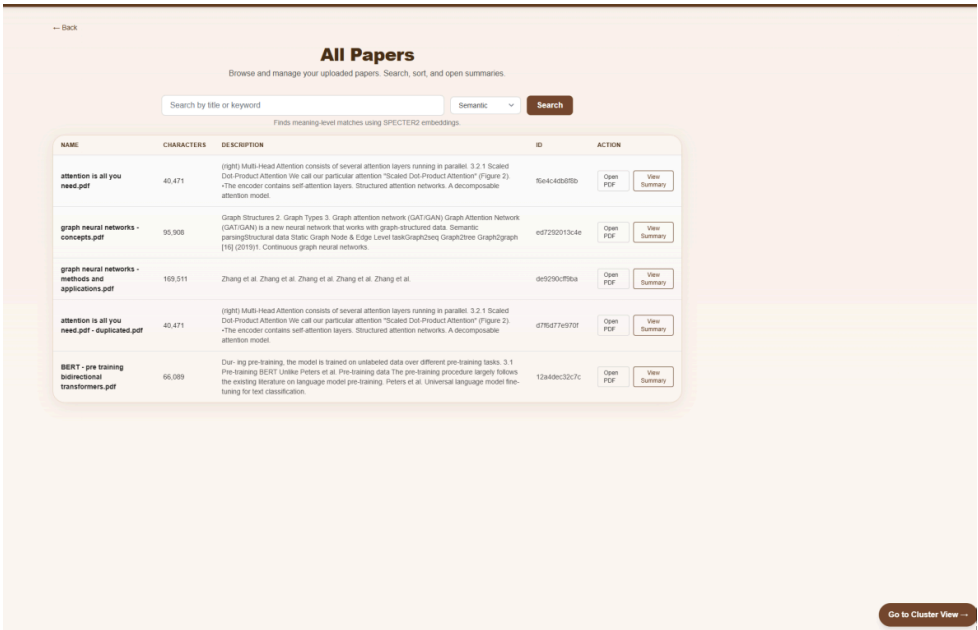


Upload interface showing active processing pipeline and real-time status feedback



Upload interface after processing completion with transition to corpus view

All Papers View	Search bar, search mode selector, table of stored papers, detail modal trigger, open PDF action, document management actions	Support scanning, retrieval, and light document management without requiring deep navigation
-----------------	--	--



All Papers view showing the default corpus layout and document structure

← Back

All Papers

Browse and manage your uploaded papers. Search, sort, and open summaries.

Semantic ▾

Finds meaning-level matches using SPECTER2 embeddings.

NAME	CHARACTERS	DESCRIPTION	ID	ACTION
attention is all you need.pdf	40,471	(right) Multi-Head Attention consists of several attention layers running in parallel. 3.2.1 Scaled Dot-Product Attention We call our particular attention "Scaled Dot-Product Attention" (Figure 2). •The encoder contains self-attention layers. Structured attention networks. A decomposable attention model.	e8de5c800e28	Open PDF View Summary
BERT - pre training bidirectional transformers.pdf	66,089	Dur- ing pre-training, the model is trained on unlabeled data over different pre-training tasks. 3.1 Pre-training BERT Unlike Peters et al., Pre-training data The pre-training procedure largely follows the existing literature on language model pre-training. Peters et al. Universal language model fine-tuning for text classification.	e9949c528992	Open PDF View Summary
graph neural networks - methods and applications.pdf	169,511	Zhang et al. Zhang et al. Zhang et al. Zhang et al. Zhang et al.	c30ebc48952a	Open PDF View Summary
attention is all you need.pdf - duplicated.pdf	40,471	No description available	c162d97f0eb2	Open PDF View Summary
attention is all you need.pdf - duplicated.pdf	40,471	(right) Multi-Head Attention consists of several attention layers running in parallel. 3.2.1 Scaled Dot-Product Attention We call our particular attention "Scaled Dot-Product Attention" (Figure 2). •The encoder contains self-attention layers. Structured attention networks. A decomposable attention model.	7d7afe1e5ef6	Open PDF View Summary
graph neural networks - concepts.pdf	95,908	No description available	78ab0c29d9768	Open PDF View Summary
attention is all you need.pdf	40,471	No description available	5244513ae15d	Open PDF View Summary
graph neural networks - methods and applications.pdf	169,511	No description available	4d57c5e40386	Open PDF View Summary
graph neural networks - concepts.pdf	95,908	Graph Structures 2. Graph Types 3. Graph attention network (GAT/GAN) Graph Attention Network (GAT/GAN) is a new neural network that works with graph-structured data. Semantic parsingStructural data Static Graph Node & Edge Level taskGraph2seq Graph2tree Graph2graph [16] (2019)! Continuous graph neural networks.	31f8a2a1433e	Open PDF View Summary
BERT - pre training bidirectional transformers.pdf	66,089	No description available	1584e38f54a0	Open PDF View Summary

Paper Detail and Recommendations Modal	Metadata panel, summary or abstract display, DOI link, open PDF action, delete control, optional external recommendations	Enable quick assessment of paper relevance and movement between related papers within the same workflow
Cluster View	Search and sort controls, cluster cards with counts and keywords, modal detail view, grouped paper listings with summaries	Surface thematic structure and support synthesis across related papers

SmartResearch

Upload

All Papers

Cluster

Export

-- Docs

Clustering

clusters are generated from your uploaded papers. keywords are pulled from the cluster's fit-att

Search by cluster name or keyword

Sort by

cluster 2

keywords: bert, token, model, pre-training, token, sentence, task, language

1 papers

keywords: bert, token, model, pre-training, token, sentence, task, language

View Papers

Graph Neural Networks: Graph Generation

keywords: graph, graphs, node, nodes, proceedings, networks, neural, graph, neural

1 papers

keywords: graph, graphs, node, nodes, proceedings, networks, neural, graph, neural

View Papers

Rock Core Orientation Technology Using Resistivity Method v1

keywords: attention, model, models, sequence, axis, part, model, item

1 papers

keywords: attention, model, models, sequence, axis, part, model, item

View Papers

Clustering

clusters are generated from your uploaded papers. keywords are pulled from the cluster's tf-idf.

Sort by ▾

cluster 2

keywords: bert, token, model, tuning, sentence, pre-training, pre-trained, cls

1 papers

keywords: bert, token, model, tuning, sentence, pre-training, pre-trained, cls

View Papers

Rock Core Orientation Technology Using Resistivity Method v1

keywords: attention, model, models, sequence, pad, anxiv, output, layer

1 papers

keywords: attention, model, models, sequence, pad, anxiv, output, layer

View Papers

Graph Neural Networks: Graph Generation

keywords: graph, graphs, proceedings, networks, node, nodes, neural, learning

1 papers

keywords: graph, graphs, proceedings, networks, node, nodes, neural, learning

View Papers

Rock Core Orientation Technology Using Resistivity Method v1

keywords: attention, model, models, sequence, pad, anxiv, output, layer

1 papers

keywords: attention, model, models, sequence, pad, anxiv, output, layer

View Papers

The Dissolution of Capital by Farooq Hamid is Licensed under a Creative Commons Attribution-Non-commercial 4.0 International License

keywords: graph, node, graph neural, neural, gnn, data, networks, nodes

1 papers

keywords: graph, node, graph neural, neural, gnn, data, networks, nodes

View Papers

Cluster view with generated groupings demonstrating variation in cluster labels and outputs

Export View	Format selection controls, inclusion toggles, export action	Generate reusable outputs for downstream review while demonstrating configurable export behaviour
-------------	---	---

Export Data

Choose what to include in your export package. Files will be generated in the selected format with optional summaries and insights.

Select what you want to export:

☒ All Papers
 ☒ Clusters List

☒ Papers By Cluster
 ☒ Selected Papers

Choose format:

☒ PDF
 ☐ CSV
 ☐ JSON

Include:

☒ Summaries
 ☒ Keywords
 ☒ Cluster Insights

Export

Export view showing configurable output options and format selection

5.2 Interface Principles

Principle	Application in SmartResearch
Simplicity	The interface follows a focused workflow rather than expanding into a complex multi-panel dashboard
Clarity	Papers, clusters, and export options are presented with minimal visual clutter to support quick understanding
Consistency	Bronze accents, rounded cards, and repeated layout patterns create a coherent visual identity across all screens
Research Fit	Tables, summaries, modal views, and export options support scanning, comparison, and revisiting information under time pressure
Progress Visibility	Upload state, search modes, result counts, and modal interactions make system state visible throughout the workflow

6. Branding and Style Guide

SmartResearch uses a restrained visual identity because it is designed to support academic screening rather than general productivity work. Users are expected to read summaries, compare metadata, explore clustered themes, and revisit papers over longer sessions, so the interface needs to feel credible, readable, and stable. A louder or more decorative style would not improve the task, because it would compete with the information itself. The branding therefore communicates clarity, structure, and trust, while the style guide ensures those qualities are applied consistently across the prototype.

Perceived visual complexity shapes how users judge and navigate digital systems, and early impressions of an interface are influenced by factors such as colourfulness and structural density (Harper et al., 2013; Reinecke et al., 2013). For SmartResearch, the visual layer is intended to support the research workflow rather than distract from it. This is especially important because the underlying task already involves dense academic material and a high cognitive demand (Sweller, 1988).

6.1 Brand Positioning and Identity

SmartResearch is positioned as a professional academic support tool rather than a consumer-facing product. Its identity is intended to feel calm, structured, and practical. The interface should suggest that the system is organised, reliable, and appropriate for serious research work. This identity is expressed less through decorative branding devices and more through visual consistency, controlled colour use, repeated layout patterns, and direct task language.

The current branding direction can be summarised through the following attributes:

Brand Attribute	Intended Effect in the Interface
Credible	The interface should look appropriate for academic and supervisory contexts.
Structured	Information should appear ordered, grouped, and easy to scan.
Readable	Dense content should remain legible across longer sessions.
Calm	The visual treatment should reduce noise rather than add to it.
Practical	Actions, labels, and workflows should feel direct and purposeful.

These attributes distinguish SmartResearch from more general note-taking or dashboard-style tools. The system is not intended to feel playful or highly commercial. It is intended to support a focused research workflow.

6.2 Visual Style Guide

The visual system uses a narrow palette built around a warm bronze accent, pale neutral surfaces, and dark text. The bronze gives the prototype a distinct identity, but it is used selectively for navigation emphasis, buttons, active states, and headings rather than dominating the entire interface. Pale backgrounds and dark text provide the main reading environment, which is important in a system built around summaries, metadata, and tables. Stronger contrast between text and background improves visual search performance and supports readability on screen (Ling & van Schaik, 2002).

Element	Current Treatment	Usage Guidance
Primary Accent	#8B5E3C	Used for navigation emphasis, primary buttons, active states, and section accents.
Deep Accent	#5E3A1C	Used for headings, strong emphasis, anchor text, and higher-contrast interface moments.
Surface	#F7F1EC	Used for cards, modals, table containers, and light grouping elements.
Background	#FFFAF6 to pale neutral	Used as the main page background to minimise visual noise.
Text	#2E221A	Used for body text, metadata, labels, and dense informational content.
Status Colours	Green, red, and amber support colours	Used for upload state, warnings, errors, and success indicators.

Typography and components follow the same restrained logic. Bold page headings establish hierarchy, while readable sans-serif body text supports quick scanning across summaries and metadata. Cards, tables, and rounded modals are used consistently because they help structure dense information without pushing the user into deeper navigation. This repeated component language helps the system feel coherent across the Upload, All Papers, Cluster, and Export views.

Element	Guidance
Typography	Bold page headings paired with readable sans-serif body text to support hierarchy and efficient on-screen scanning.
Component Style	Simple cards, clean tables, rounded modals, restrained shadows, and generous spacing to keep dense information manageable.
Layout Behaviour	A shallow four-view workflow with progressive detail through modals, search states, and export options.
Iconography	Minimal, utilitarian icons that reinforce action rather than decorate the interface.
Content Structure	Summary-first presentation, labelled clusters, and narrow task wording to support quick judgement and retrieval.
Tone	Professional, academic, structured, and interpretable rather than playful or promotional

The overall tone of the interface is direct and practical. Labels such as “Upload,” “All Papers,” “Cluster,” and “Export” reflect the user’s actual workflow without relying on vague or overly branded language. This is appropriate for a research tool, where clarity matters more than personality. In SmartResearch, branding is reflected not only in colour or layout, but also in whether the interface feels consistent, legible, and fit for purpose.

6.3 Application to the Prototype

The prototype reflects this branding direction through the consistent use of bronze-accent navigation and interactive elements, pale surfaces, and repeated card-based layouts. The Upload, All Papers, Cluster, and Export screens follow the same visual logic, which helps the product read as one system rather than a set of disconnected screens. This consistency supports usability by reducing disorientation and maintaining a stable visual structure across the workflow.

The interface style also aligns with the practical demands of document-heavy academic work. Summaries, metadata, clusters, and export controls are presented in a way that supports scanning and retrieval rather than decoration. The result is a product identity grounded in clarity, coherence, and research fit.

\

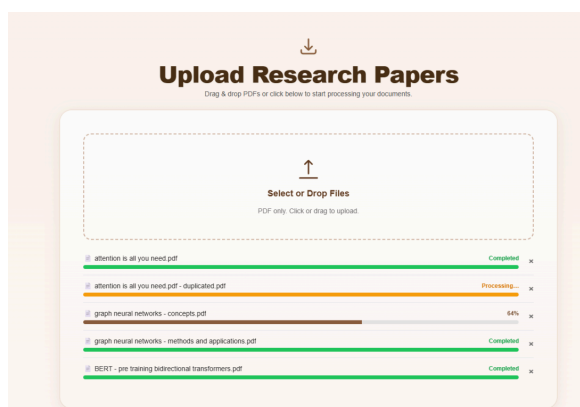
7. Functionality and User Interaction

SmartResearch is designed as a linear, low-friction research workflow that supports document ingestion, corpus exploration, retrieval, thematic synthesis, and export within a small number of tightly connected views. The system uses a React frontend connected to a FastAPI backend, with local storage handling uploaded papers, extracted text, metadata, and generated outputs.

SmartResearch is designed for students and early-stage researchers who are required to manage, review, and synthesise multiple academic papers within a limited timeframe. These users often encounter information overload, fragmented organisation, and difficulty identifying relationships between sources. The system is structured to reduce organisational overhead, streamline retrieval, and support higher-level synthesis rather than isolated reading.

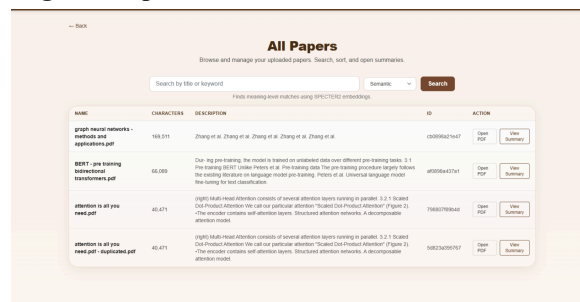
The interaction model prioritises clarity and cognitive load reduction by guiding users through a structured sequence rather than exposing all functionality at once. Secondary actions such as summary viewing, metadata inspection, PDF access, deletion, and recommendations are embedded within each stage to maintain workflow continuity without forcing users to navigate away from their current task.

7.1 Primary Interaction Sequence



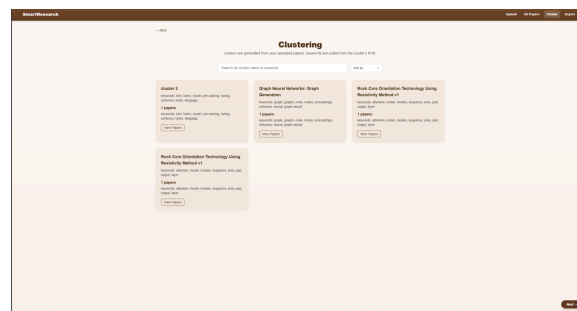
Upload workflow showing active processing and system feedback

The primary interaction sequence begins in the Upload view, where the user adds one or more academic PDF files to the system. This entry point ensures that all documents move through a consistent processing pipeline, reducing ambiguity and preventing fragmented data states. The frontend passes uploaded files to the backend, where they are stored locally and processed through text extraction, OCR fallback, metadata handling, summary generation, semantic indexing, and cluster preparation. This automated pipeline reduces manual preprocessing and allows users to begin interacting with their documents quickly. **The full workflow is illustrated in the activity diagram provided in appendix A, which shows the complete interaction sequence across upload, processing, retrieval, clustering, and export.**



All Papers view providing access to the processed document corpus

Once processing is complete, the user moves to the All Papers view, which acts as the central hub of the system. Consolidating all documents into a single searchable corpus helps avoid fragmentation and supports both targeted retrieval and exploratory browsing without requiring context switching. The availability of keyword, semantic, and hybrid search modes supports different research behaviours typical of students and researchers working across both familiar and unfamiliar topics (*see Appendix A, Figures A.1 and A.2*). Document-level actions such as viewing summaries, inspecting metadata, accessing PDFs, and deletion are integrated directly into this view to minimise navigation overhead.



Cluster view supporting thematic exploration of grouped documents

After initial exploration, the user transitions to the Cluster view, which shifts interaction from individual document inspection to thematic analysis. Grouping papers into clusters allows users to identify relationships, recurring topics, and conceptual groupings across the corpus, supporting synthesis rather than isolated evaluation. This is particularly relevant in literature review tasks, where understanding connections between sources is more valuable than analysing each paper independently.

The final stage is the Export view, where the user selects relevant content and generates an output for external use. Positioning export at the end of the workflow reinforces a clear progression from ingestion to synthesis and helps ensure that exported material reflects a considered review process (*see Appendix A, Figure A.3*).

7.2 Primary Scenario Walkthrough

A typical user begins with a collection of academic papers on a new topic and uploads them through the Upload view. The system processes these files automatically, removing the need for manual organisation and enabling the user to begin interacting with the material shortly after upload.

The user then moves to the All Papers view, where they review the processed corpus and use search tools to locate relevant material. They may begin with semantic search to identify conceptually related papers, then refine results using keyword or hybrid search depending on their familiarity with the topic. When a paper appears relevant, the user opens its detail view to inspect summaries, metadata, and identifiers such as DOI, and may access the original PDF for deeper reading.

After identifying relevant papers, the user moves to the Cluster view to explore thematic groupings within the corpus. This allows them to understand broader patterns and relationships, supporting synthesis across sources rather than isolated evaluation. The cluster-based interaction helps the user move from collection to comprehension, which is important in producing structured academic work.

Finally, the user proceeds to the Export view and generates an output containing selected papers, summaries, or grouped insights. This output can then be used for writing, reporting, or submission. Through this sequence, SmartResearch supports a complete workflow that prioritises efficiency, clarity, and synthesis from initial upload through to final output.

8. Prototype Progress

The SmartResearch prototype now demonstrates a working implementation of the core system workflow. The current build includes a routed React frontend, a FastAPI backend, local file-based storage, and integrated support for document upload, text extraction, metadata handling, search, clustering, and export.

The frontend includes the main workflow views for Upload, All Papers, Cluster, and Export. These views are connected through the application router and reflect the shallow navigation structure used throughout the system. The Upload view supports drag-and-drop file handling and visible upload state, the All Papers view supports corpus browsing and multiple search modes, the Cluster view supports grouped thematic exploration, and the Export view provides output controls. Together, these views create a connected and usable interface across the main workflow.

The backend supports a substantial portion of the planned functionality. The FastAPI application includes endpoints for health checking, document upload, document retrieval, metadata refresh, clustering, keyword search, semantic search, hybrid search, similar-document retrieval, external recommendations, reindexing, PDF serving, and export generation. The backend also includes OCR fallback for low-text PDFs, summary generation, metadata enrichment, semantic indexing, and local persistence for uploaded files and generated outputs.

The system demonstrates the expected workflow of uploading papers, processing them, browsing stored results, searching the corpus, exploring clusters, and generating export output. Some features are more mature than others, and a small number of interface behaviours still require refinement to make the overall experience fully consistent.

The remaining work is focused on improving reliability and consistency across the existing workflow. This includes resolving display issues, tightening alignment between frontend controls and backend support, and ensuring that the final demonstrated path through the system is stable and clear.

Prototype Area	Current Evidence	Status
Frontend Workflow Views	Upload, All Papers, Cluster, and Export views are implemented as routed pages in the React frontend	Implemented
Upload and Ingestion	Users can upload PDF files through the interface, and the backend processes them through the document pipeline	Implemented
Text Extraction and OCR Fallback	The backend extracts text from PDFs and includes OCR fallback for low-text or scanned documents	Implemented
Metadata Handling	Metadata is extracted, stored locally, and refreshed through dedicated backend logic and endpoints	Implemented
Search	The prototype supports keyword, semantic, and hybrid search through integrated frontend and backend behaviour	Implemented
Clustering	Papers can be grouped into clusters for thematic exploration, with cluster data exposed through the backend and shown in the interface	Implemented

Paper Detail Interaction	Paper-level detail access, summary viewing, PDF opening, deletion, and recommendation-related actions are present, but some display behaviour still requires refinement	Implemented / In Progress
Export	The prototype includes an export interface and backend export generation, but some interface options are broader than current backend support	Implemented / In Progress
Local Storage and Persistence	Uploaded PDFs, extracted text, metadata, and generated outputs are stored locally through a file-based persistence model	Implemented
Prototype Stability and Polish	The core workflow is present, but some edge cases, consistency issues, and final presentation details still require cleanup	In Progress

9. Current Limitations and Next Iteration Work

The current SmartResearch system demonstrates the intended workflow for screening and organising academic papers, but several areas still require refinement. These limitations are less about the overall structure of the system and more about consistency, testing, and feature maturity. Upload, browsing, search, clustering, and export are all present within the connected frontend and backend workflow, but some features are more reliable than others, and not all outputs are yet consistent enough to be treated as fully polished.

One limitation is the uneven maturity of some implemented features. While the main views are in place and connected, certain interface behaviours and data displays still need refinement so that results appear consistently across screens. Summary display, detailed paper interaction, and document-level actions require further cleanup to ensure that metadata, summaries, and related controls are shown reliably for each uploaded paper. Export is also part of the current workflow, but its behaviour would benefit from clearer handling and closer alignment between the interface and backend support.

Another limitation is the variability of system-generated outputs. SmartResearch can generate summaries, metadata, and clustered groupings, but the quality of these outputs depends on the structure and quality of the uploaded PDFs. This is particularly relevant for OCR fallback, metadata enrichment, and clustering, where the usefulness of the result is affected by the accuracy of extracted text and the consistency of source material. The system is therefore best understood as a support tool for screening and organisation rather than a substitute for human judgement.

The current design also uses a lightweight local storage and processing model. This suits the project scope and keeps the system manageable, but it also limits scalability. The system is appropriate for bounded document sets and demonstration use, but it has not yet been validated for larger corpora, heavier usage, or broader performance demands. In addition, while the core workflow remains local, some recommendation or enrichment features may rely on optional external services, so those boundaries should be communicated clearly.

The next stage of work is focused on improving what already exists rather than expanding the scope of the system. This includes refining interface consistency, resolving display issues, validating summary and clustering quality on a bounded paper set, and ensuring the overall workflow is stable enough for demonstration and assessment. Beyond that, further work would improve export behaviour, expand testing, strengthen metadata accuracy, and increase the reliability of paper-level detail and recommendation features.

Current Limitation	Next Iteration Work
Some frontend behaviours and displays are not yet fully consistent across all views	Refine frontend-backend integration so summaries, metadata, and document actions are displayed more reliably
Export functionality is implemented but still limited in maturity	Expand and stabilise export options, improve format handling, and align interface choices with backend support
Summary, metadata, and clustering quality can vary depending on input documents	Run bounded evaluation on a representative paper set and refine output handling where quality is inconsistent
OCR and metadata enrichment depend on the quality and structure of uploaded PDFs	Improve fallback handling, error reporting, and validation for lower-quality or less predictable source documents
The system has been developed and tested within a limited project scope	Extend testing across more papers, edge cases, and workflow scenarios to improve reliability

Core processing is local, but optional external enrichment may create ambiguity	Clarify which functions are fully local and which rely on external services, and present those features more transparently in the interface
The current system is suitable for demonstration and bounded use, but not yet for large-scale deployment	Explore scalability improvements, stronger persistence options, and broader performance validation in later development stages

Overall, the next stage of SmartResearch is centred on refinement rather than major redesign. The core workflow and architectural direction are already in place. The remaining work is to improve reliability, tighten presentation, and increase the maturity of the existing feature set so that the system more clearly reflects its intended use as a practical literature review support tool.

10. Project Risks and Progress Management

The main risks at this stage are practical rather than conceptual. SmartResearch has a defined direction and a working core workflow, so the primary concern is whether the system can be presented as stable, consistent, and clearly aligned across the report, prototype, and demonstration. The focus has shifted away from new feature development towards reliability, testing, and overall polish. Remaining risks relate to edge-case handling, output consistency, and ensuring the system behaves consistently across all core features.

Risk	Impact	Mitigation Now in Use	Remaining Action
Feature Instability Across the Workflow	High	The main workflow for upload, browsing, clustering, search, and export is implemented	Test the full user flow end-to-end and resolve any breaks, missing data, or inconsistencies between screens
Mismatch Between Report, Prototype, and Demo	High	Documentation has been updated to reflect the current system	Keep terminology, screenshots, and feature descriptions consistent across all submission components
Inconsistent Summary or Cluster Quality	Medium	Current outputs are generated using existing summarisation and clustering pipelines	Run a small set of test papers and document strengths and limitations of outputs clearly
Unfinished Interface Polish	Medium	Core interface structure and main views are functional	Refine layout consistency, wording, and display issues rather than expanding feature scope
Time Pressure Before Submission	High	Core functionality is already implemented	Freeze scope and prioritise testing, screenshots, cleanup, and final packaging
Team Coordination in the Final Stage	Medium	Shared repository and supporting documentation are in place	Maintain clear responsibility for testing, report editing, screenshots, and submission tasks
Privacy and Expectations Around External Features	Medium	Core workflow is based on local processing and storage	Clearly distinguish between local functionality and optional external features such as recommendations

11. Conclusion

SmartResearch has moved from an initial design into a working system that demonstrates the core workflow of screening and organising academic papers. The current implementation supports document upload, metadata extraction, summary generation, semantic search, clustering, and local storage, with the frontend and backend integrated across the main user flow. This allows the system to reflect the research process in a way that is usable and testable, rather than remaining at the level of design.

The system prioritises a clear and consistent workflow over unnecessary complexity. Upload, browsing, clustering, and export are structured as connected stages, with supporting interactions such as search modes, metadata inspection, and document-level actions embedded within each view. This keeps the interface focused while still allowing users to move between individual papers and broader thematic groupings.

Some areas still require refinement, particularly around summary consistency, clustering quality, performance validation, and handling of edge cases. These do not affect the overall structure of the system, but they influence the reliability and polish of the outputs.

The remaining work is centred on stabilising and validating the existing workflow. This includes ensuring consistent behaviour across features, improving output quality, refining interface interactions, and completing partially implemented functionality such as export. The aim is to present a system that is not only functional, but also stable, coherent, and aligned with its intended use.

Overall, SmartResearch delivers a structured and practical approach to literature review tasks, supported by a working implementation that makes the design decisions visible in use.

Appendix A. Additional Interface Screens

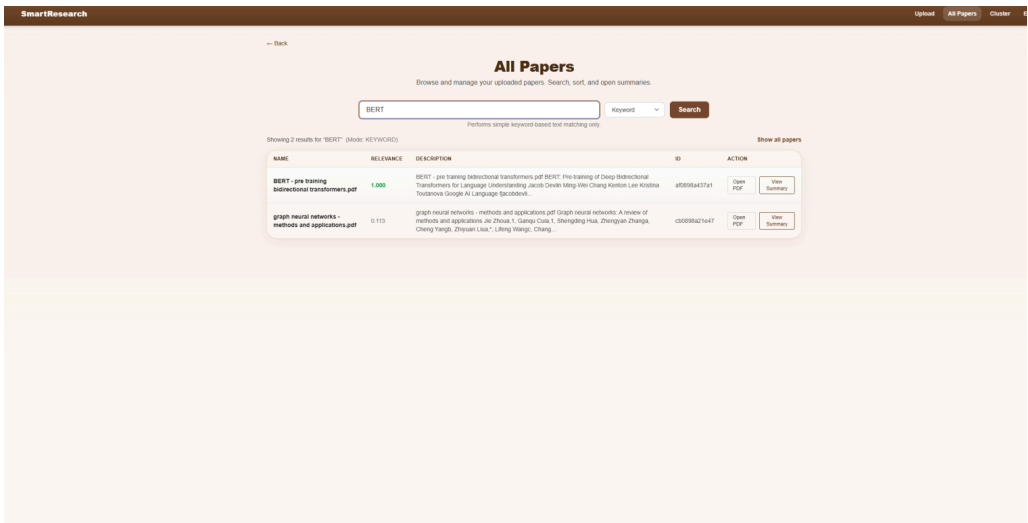


Figure A.1 Keyword search results using text-based matching

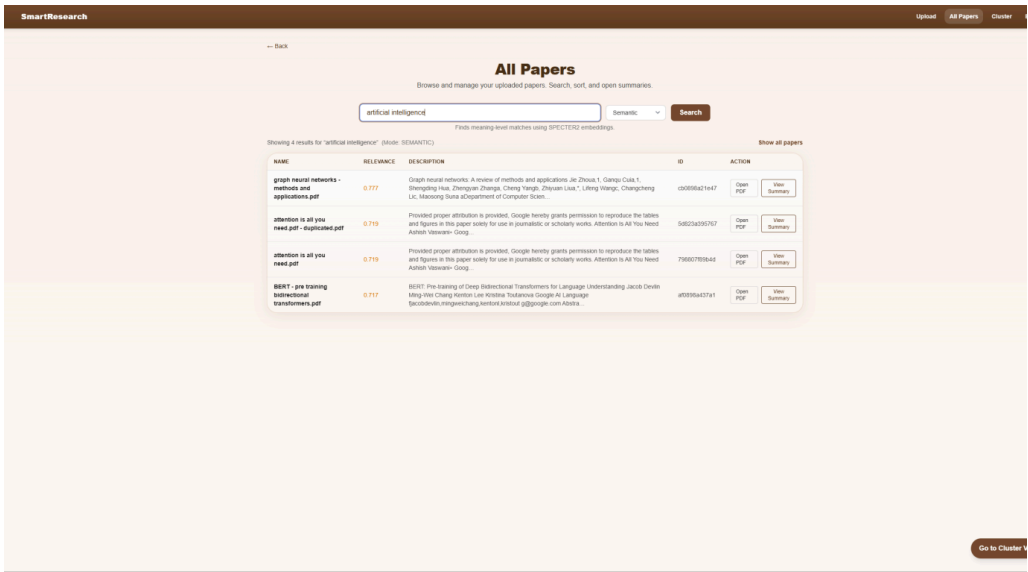


Figure A.2 Semantic search results using embedded-based retrieval

Export Data

Choose what to include in your export package. Files will be generated in the selected format with optional summaries and insights.

Select what you want to export:

☒ All Papers

☒ Clusters List

☒ Papers By Cluster

☒ Selected Papers

Choose format:

☒ PDF

☐ CSV

☐ JSON

Include:

☒ Summaries

☒ Keywords

☒ Cluster Insights

Export

Figure A.3 Export interface showing configurable output options

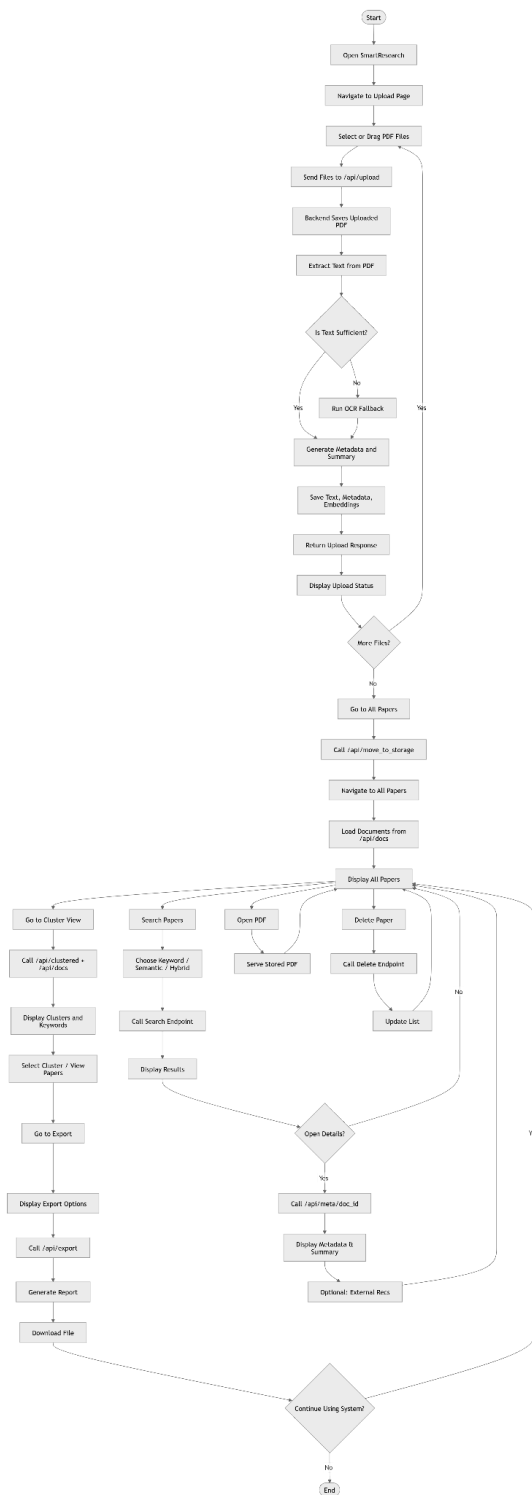
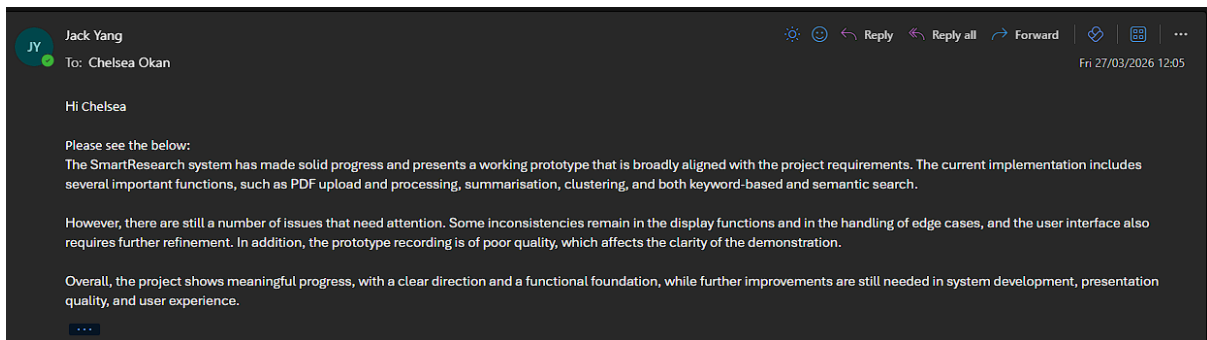


Figure A.4 Activity diagram showing the end-to-end smartresearch workflow, including document upload, processing (text extraction with ocr fallback, metadata and summary generation, indexing), and user interaction across search, clustering, and export views.

Appendix B. Supervisor Progress/Comments



References

- Harper, S., Jay, C., Michailidou, E., & Quan, H. (2013). Analysing the visual complexity of web pages using document structure. *Behaviour & Information Technology*, 32(5), 491-502.
- Ling, J., & van Schaik, P. (2002). The effect of text and background colour on visual search of Web pages. *Displays*, 23(5), 223-230.
- Reinecke, K., Yeh, T., Miratrix, L., Mardiko, R., Zhao, Y., Liu, J., & Gajos, K. Z. (2013). Predicting users' first impressions of website aesthetics with a quantification of perceived visual complexity and colorfulness. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 2049-2058).
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257-285.